Matemática para Aprendizaje Automático

Hoja de Ejercicios - Unidad 1 - Semana 4

Universidad Católica del Uruguay • 2026

Costo Computacional y Precisión Numérica

Análisis de Costo Computacional

Problema 1.4.1 — Costo de Operaciones Matriciales

Considera las siguientes operaciones matriciales:

1. Halle el costo computacional de multiplicar dos matrices rectangulares de tamaño $m \times n$ y $n \times p$.
2. Considere el producto de tres matrices

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times p}, \quad C \in \mathbb{R}^{p \times q}$$

Suponiendo que $m > n, \quad p > q$, ¿En qué orden deberías multiplicarlas para minimizar el número de operaciones?
Estime la cantidad de operaciones necesarias.

3. Muestre que el producto de una matriz A de tamaño $n \times n$ con su transpuesta puede computarse con menos operaciones que el producto de dos matrices arbitrarias del mismo tamaño. Estime la cantidad de operaciones necesarias.

Problema 1.4.2 — Optimización de Algoritmos

Considere el siguiente algoritmo para resolver un sistema triangular inferior $Lx = b$

```
def solve_triangular(L, b):
    n = len(b)
    x = np.zeros(n)

    for i in range(n):
        x[i] = b[i]

        for j in range(i):
            x[i] -= L[i,j] * x[j]

        x[i] /= L[i,i]

    return x
```

1. Muestre que este algoritmo requiere aproximadamente $\frac{n^2}{2}$ operaciones.
2. ¿Podrías modificar el algoritmo para que sea más eficiente en algún sentido? Sugerencia: ¿Puede eliminarse algún bucle?

Verifique el funcionamiento (¿da el mismo resultado?) y eficiencia (¿demora menos?) del algoritmo modificado implementándolo en Python.

Precisión Numérica y Condicionamiento

Problema 1.4.3 — Análisis de Errores en Sistemas Lineales

Considere los siguientes sistemas lineales de la forma $Ax = b$:

Sistema 1

$$\begin{bmatrix} 1.0000 & 2.0000 \\ 2.0000 & 4.0001 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3.0000 \\ 6.0001 \end{bmatrix}$$

Sistema 2

$$\begin{bmatrix} 1.0000 & 2.0000 \\ 2.0000 & 4.0001 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3.0000 \\ 6.0002 \end{bmatrix}$$

1. Muestre que la solución al primer problema es $x_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ y al segundo problema es $x_2 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$
2. Calcule la sensibilidad absoluta y relativa del problema.
3. Calcule el número de condición de la matriz A. Muestre que la sensibilidad relativa del problema es del orden de (pero menor que) este valor.

Problema 1.4.4 — Análisis de Errores en Sistemas Lineales

El problema anterior muestra la sensibilidad del sistema ante cambios en el término independiente pero el problema es igualmente sensible ante cambios en A. Considere ahora los siguientes sistemas lineales de la forma $Ax = b$:

Sistema 1

$$\begin{bmatrix} 1.0000 & 2.0000 \\ 2.0000 & 4.0001 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3.0000 \\ 6.0001 \end{bmatrix}$$

Sistema 2

$$\begin{bmatrix} 1.0000 & 2.0000 \\ 2.0000 & 4.0002 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3.0000 \\ 6.0001 \end{bmatrix}$$

1. Muestre que la solución al primer problema es $x_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ y al segundo problema es $x_2 = \begin{bmatrix} 2 \\ 0.5 \end{bmatrix}$
2. Calcule la sensibilidad absoluta y relativa del problema.
3. Muestre que la sensibilidad relativa del problema es del orden de (pero menor que) este valor.

Problema 1.4.5: Análisis de Errores en Cálculos de Autovalores

Considere las matrices del ejercicio anterior.

1. Halle sus valores propios.
2. Calcule la sensibilidad relativa del menor valor propio y muestre que también es del orden del número de condición de las matrices.

Estabilidad

Cuando un algoritmo $\bar{f}$ es estable para resolver un problema $f(x)$, esperamos que el error de estimar f esté dado solamente por el número de condición del problema. Específicamente, si se utiliza representación de punto flotante, esperamos que el error relativo de la estimación satisfaga que:

$$\frac{\|\bar{f}(x) - f(x)\|}{\|f(x)\|} = O(\kappa(f(x))\,\epsilon)$$

Donde ϵ es el épsilon de máquina (error esperado en la representación de x) y κ es el número de condición del problema $f(x)$.
Se recuerda que para un problema diferenciable, el número de condición está dado por:

$$\kappa(f(x)) = \frac{\|x\|}{\|f(x)\|}\,\|f'(x)\|$$

Problema 1.4.6 — Estabilidad Algorítmica en el Cálculo de la Exponencial

Considere el cálculo de

$$f(x) = e^x$$

1. Mediante diferenciación, halle el número de condición del problema cuando $x = -20$ y cuando $x = 20$

   Considere ahora el siguiente método para calcular e^x mediante el polinomio de Taylor de dicha función: $e^x \approx \sum_{i=0}^{n} \frac{x^i}{i!}$ hasta orden 10.

```
import math
import numpy as np

def exp_unstable(x):
    # Calcula e^x usando la serie de Taylor directamente
    result = 0

    for i in range(10):
        result += x**i / math.factorial(i)

    return result
```

2. Implemente este algoritmo y úselo para calcular e^{20} y e^{-20}. Compare los resultados obtenidos con los verdaderos valores: $e^{20} \approx 2423600.2$ y $e^{-20} \approx 4 \times 10^{-7}$ ¿Qué ocurre? ¿Cómo se compara el error relativo en el cálculo de ambos valores con el número de condición? ¿Por qué?

Considere ahora el algoritmo alternativo que se da a continuación, que en el caso de $x < 0$, estima primero e^{-x} (también mediante su polinomio de Taylor de orden 10), y luego calcula el recíproco:

```python
def exp_stable(x):
    # Calcula e^x usando la serie de Taylor con términos positivos y negativos
    if x < 0:
        return 1 / exp_stable(-x)

    result = 0

    for i in range(10):
        result += x**i / math.factorial(i)

    return result
```

3. Repita la parte anterior con este algoritmo. Vea que esta es una implementación estable para todo x.

Problema 1.4.7: Estabilidad en el cálculo de valores propios

Considere el cálculo de los valores propios de la siguiente matriz:

$$\begin{bmatrix} 1+10^{-14} & 0 \\ 0 & 1 \end{bmatrix}$$

1. Calcule los valores propios de esta matriz usando np.linalg.eigvals().
2. Repita el cálculo mediante el cálculo del polinomio característico de la matriz y sus raíces. ¿Qué ocurre?
3. Concluya acerca de la estabilidad de ambos algoritmos.

Problema 1.4.8: Condicionamiento de las raíces de un polinomio

La inestabilidad vista en el problema anterior es generada por elegir una mala estrategia para la resolución del problema (polinomio característico + raíces). Específicamente, la segunda etapa del algoritmo genera una inestabilidad pues es un problema mal condicionado. El siguiente ejercicio ilustra el problema.

1. Considere el polinomio cuadrático $x^2 + bx + c$. Utilizando la conocida fórmula para las raíces de estos polinomios en función de los coeficientes b y c, halle el número de condición del problema de hallar una de las raíces (cualquiera) con respecto al coeficiente c.
2. En el caso de que $b^2 \approx 4c$ (es decir, cuando la raíz es real doble), el número de condición del problema es arbitrariamente grande.
3. Concluya acerca de la precisión relativa de resolver el problema de hallar las raíces del polinomio, y su impacto cuando un algoritmo depende de hallar dichas raíces como en el problema anterior.

Problema 1.4.9: Estabilidad en Raíces de Polinomios

Si bien el resultado anterior se mostró específicamente para polinomios de segundo orden que están "cerca" de tener raíces dobles, el mismo es un resultado general acerca del problema de hallar las raíces de un polinomio. Un ejemplo clásico de la sensibilidad de las raíces de un polinomio en un caso en que el polinomio no está "cerca" de tener raíces dobles es el polinomio de Wilkinson:

$$p(x) = (x-1)(x-2)(x-3)\cdots(x-20) = x^{20} - 210x^{19} + \cdots + 20!$$

1. Muestre que las raíces exactas de este polinomio son los números enteros del 1 al 20.

2. Implemente en Python el cálculo de los coeficientes del polinomio de Wilkinson con el siguiente código:

```python
import numpy as np
import sympy

def wilkinson_coefficients(n):
    """
    Calculates the coefficients of the Wilkinson polynomial of degree n.

    Args:
        n: The degree of the Wilkinson polynomial.

    Returns:
        A NumPy array containing the coefficients of the polynomial.
    """

    x = sympy.Symbol('x')
    polynomial = sympy.prod(x - i for i in range(1, n + 1))
    coefficients = sympy.Poly(polynomial, x).all_coeffs()

    return np.array(coefficients, dtype=np.float64)
```

3. Calcule las raíces del polinomio de Wilkinson usando np.roots() y observe los resultados. Compare con la parte 1.

4. Repita la parte anterior pero utilizando punto flotante de 32 bits en lugar de 64 para guardar los coeficientes del polinomio antes de calcular sus raíces. ¿Qué ocurre en este caso? Concluya acerca del condicionamiento del problema de hallar las raíces del polinomio (asuma que np.roots() utiliza un algoritmo estable).

Por más información acerca del polinomio de Wilkinson, ver Wilkinson's polynomial en Wikipedia.